SPGraph — Análise Territorial e Acessibilidade em Saúde Pública por Meio de Teoria dos Grafos

> **Modelagem Computacional da Rede de Atenção Primária e Hospitalar no Município de São Paulo**
> **Conformidade com a ODS 10 — Redução das Desigualdades (ONU)**

Equipe de Desenvolvimento e Créditos Acadêmicos

- **Lucas Fernandes de Camargo** — RA 10419400
- **Lendy Naiara Carpio Pacheco** — RA 10428525
- **Anna Luiza Stella Santos** — RA 10417401

Projeto Prático Integrado de Teoria dos Grafos & Desenvolvimento de Sistemas

---

1. Resumo Executivo e Contextualização do Problema

1.1 O Desafio Urbano de São Paulo

A cidade de São Paulo é a maior metrópole da América Latina, abrigando mais de 11,4 milhões de habitantes distribuídos em 96 distritos administrativos. A distribuição espacial dos serviços públicos de saúde (Unidades Básicas de Saúde — UBS, Unidades de Pronto Atendimento — UPA e Hospitais SUS) apresenta

marcantes disparidades territoriais. Enquanto a região central e parte da zona oeste contam com alta densidade de equipamentos, os distritos periféricos das zonas Leste, Norte e Sul frequentemente enfrentam maiores distâncias de deslocamento e tempos de viagem até a referência de saúde mais próxima.

1.2 Objetivo Geral

Desenvolver um sistema computacional funcional e interativo baseado em **Teoria dos Grafos** para modelar, quantificar, analisar e visualizar as desigualdades no acesso aos serviços públicos de saúde no município de São Paulo, fornecendo suporte quantitativo à tomada de decisão e planejamento urbano.

1.3 Objetivos Específicos

1. **Modelar** a rede de saúde urbana como um grafo ponderado e não-orientado ($|V| = 71$, $|E| = 215$).
2. **Implementar** algoritmos clássicos de grafos (Dijkstra para menores caminhos, BFS para alcance topológico e métricas de centralidade).
3. **Desenvolver** um índice sintético de cobertura territorial ($S_i \in [0, 100]$) que mensure a acessibilidade de cada nó da rede.
4. **Disponibilizar** uma arquitetura dupla de interação:
   - **Interface Web Interativa (Streamlit + Plotly + KML)** para análise visual e navegação em mapas geográficos.
   - **Interface CLI de Terminal (Python)** compatível com a especificação padronizada `grafo.txt` (Opções de menu `a` até `j`).
5. **Avaliar** a usabilidade da solução sob as Heurísticas de Nielsen (Interação Humano-Computador — IHC).

---

2. Fundamentação Matemática e Modelagem em Teoria dos Grafos

2.1 Formalização do Grafo G

O sistema de transporte e acesso à saúde é formalizado como uma quádrupla:

$$G = (V, E, W_V, W_E)$$

Onde:

- V **(Conjunto de Vértices):** Representa os distritos/equipamentos de saúde da rede (n = 71 vértices).
- E **(Conjunto de Arestas):** Representa as ligações físicas/geográficas de adjacência distrital entre os nós (m = 215 arestas).
- $W_V : V \to \mathbb{R}^+$ **(Pesos dos Vértices):** População residente associada a cada nó (dados reais do IBGE/MSP).
- $W_E : E \to \mathbb{R}^+$ **(Pesos das Arestas):** Distância em quilômetros (km) estimada entre os centros geográficos dos distritos conectados.

```
[ Vértice u: UBS/Distrito A ]
 População: W_v(u)
            \
             \  Aresta e = (u, v)
              \ Peso W_e(e) = Distância (km)
               \
[ Vértice v: UBS/Distrito B ]
 População: W_v(v)
```

2.2 Especificação do Tipo de Grafo (`Tipo 3`)

No padrão de serialização do projeto (`grafo.txt`), o grafo é classificado sob o **Tipo 3**:

- **Orientação:** Não-orientado (as arestas permitem tráfego bidirecional de pacientes, de modo que $w(u, v) = w(v, u)$).

- **Ponderação nos Vértices:** Sim (peso = População distrital real).
- **Ponderação nas Arestas:** Sim (peso = Distância física em km).

2.3 Estudo de Conexidade e Grafo Reduzido

- **Conexidade:** O grafo G construído é **totalmente conexo** (Componentes Conexas $= 1$). Isso garante que existe pelo menos um caminho viável entre qualquer par de nós da rede $(u, v) \in V \times V$.
- **Densidade do Grafo:**

$$\text{Densidade} = \frac{2 \cdot |E|}{|V| \cdot (|V| - 1)} = \frac{2 \cdot 215}{71 \cdot 70} = \frac{430}{4970} \approx 0{,}0865$$

  O grafo apresenta densidade de $8{,}65\%$, caracterizando uma estrutura **esparsa e realista** de redes de transporte urbano.
- **Grafo Reduzido:** Por ser um grafo não-orientado e conexo, a partição em componentes fortemente conexas resulta em um único componente, tornando o grafo reduzido um grafo trivial com 1 nó.

---

3. Algoritmos Computacionais de Grafos

3.1 Algoritmo de Dijkstra (Menor Caminho Ponderado)

Utilizado para encontrar a rota mínima de menor custo em distância entre uma origem $u \in V$ e um destino $v \in V$, ou para determinar a menor distância da origem a todos os serviços de saúde disponíveis.

Formulação Matemática:

Dado um vértice de origem s, a distância $d(v)$ para todo $v \in V$ satisfaz a propriedade de relaxamento:

$$d(v) \leq d(u) + w(u, v), \quad \forall (u, v) \in E$$

Pseudo-código Simplificado:

```
def dijkstra(G, origem):
    distancia = {v: float('inf') for v in G.nodes}
    distancia[origem] = 0
    pq = MinPriorityQueue()
    pq.insert(origem, 0)

    while not pq.is_empty():
        u = pq.extract_min()
        for v, peso in G.neighbors_with_weight(u):
            if distancia[u] + peso < distancia[v]:
                distancia[v] = distancia[u] + peso
                pq.decrease_key(v, distancia[v])

    return distancia
```

Complexidade de Tempo:

Com implementação baseada em Fila de Prioridade (Heap Binário):

$$\mathcal{O}((|V| + |E|) \log |V|)$$

Dado $|V| = 71$ e $|E| = 215$, a execução ocorre em fração de milissegundos ($< 1\ \text{ms}$).

3.2 Algoritmo BFS (Breadth-First Search — Busca em Largura)

Utilizado para analisar a topologia de vizinhança e determinar a acessibilidade em termos de número de "saltos" (profundidade relacional) a partir de um distrito inicial, sem considerar o peso das arestas.

Função no Projeto:

- Determina a árvore de alcance territorial (camadas de 1 a k saltos de distância topológica).

- Avalia a redundância de rotas e a resiliência da rede em caso de bloqueio de nós.
- **Complexidade:** $\mathcal{O}(|V| + |E|)$.

3.3 Métricas de Centralidade de Grafos

Para identificar os nós mais estratégicos e gargalos no fluxo de pacientes, foram calculadas três métricas de centralidade:

1. Centralidade de Grau (*Degree Centrality*)

Mede o número direto de conexões (vizinhos geográficos) de cada nó:

$$C_D(v) = \frac{\text{grau}(v)}{|V| - 1}$$

- **Nós de maior grau:** `UBS JARDIM JAQUELINE` (Grau 11), `UBS CHÁCARA DO CONDE` (Grau 11), `UBS GAIVOTAS` (Grau 11), `UBS DONA ADELAIDE LOPES` (Grau 11).

2. Centralidade de Proximidade (*Closeness Centrality*)

Mede a eficiência de um nó em alcançar todos os outros nós da rede via menores caminhos ponderados:

$$C_C(v) = \frac{|V| - 1}{\sum_{u \neq v} d(v, u)}$$

- **Nós de maior proximidade:** `UBS BUTANTÃ` $(0{,}0735)$, `UBS CHÁCARA DO CONDE` ($0{,}0735$), `UBS JARDIM JAQUELINE` $(0{,}0727)$.

3. Centralidade de Intermediação (*Betweenness Centrality*)

Mede a frequência com que um nó aparece nos caminhos mínimos entre outros pares de nós da rede:

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

- **Nós de maior intermediação (nós-ponte):** `UBS LUAR DO SERTÃO` $(0{,}3107)$, `UBS CHÁCARA DO CONDE` $(0{,}2195)$, `UBS JARDIM OLINDA` $(0{,}2128)$.
- *Interpretação:* `UBS LUAR DO SERTÃO` atua como um ponto crítico de articulação entre subsetores periféricos e o restante da rede.

---

4. Métricas de Acessibilidade e Cobertura Territorial

4.1 Índice Sintético de Cobertura (S_i)

Para transformar as distâncias contínuas do algoritmo de Dijkstra em um indicador compreensível pelo público e pelos gestores públicos, formulou-se o **Score de Cobertura Territorial (S_i)**:

$$S_i = \max\left(0,\ \left(1 - \frac{d_i}{d_{\max}}\right) \times 100\right)$$

Onde:

- d_i: Distância mínima (em km) do nó i até o serviço de saúde mais próximo do tipo selecionado.
- $d_{\max} = 20{,}0\ \text{km}$: Distância limite de corte territorial.
- **Leitura do Score:**
  - $S_i = 100$: O serviço de saúde está localizado **no próprio distrito**.
  - $0 < S_i < 100$: O serviço está disponível a uma distância inter-distrital d_i.
  - $S_i = 0$: O serviço está a $20\ \text{km}$ ou mais (área crítica de desassistência).

4.2 Resultados Comparativos por Equipamento de Saúde

Com base no banco de dados real processado (`deinfosacadsau2014.csv` e `evolucao_msp_pop_sexo_idade.csv`), foram obtidas as seguintes estatísticas

consolidadas para os 71 distritos/UBSs analisados:

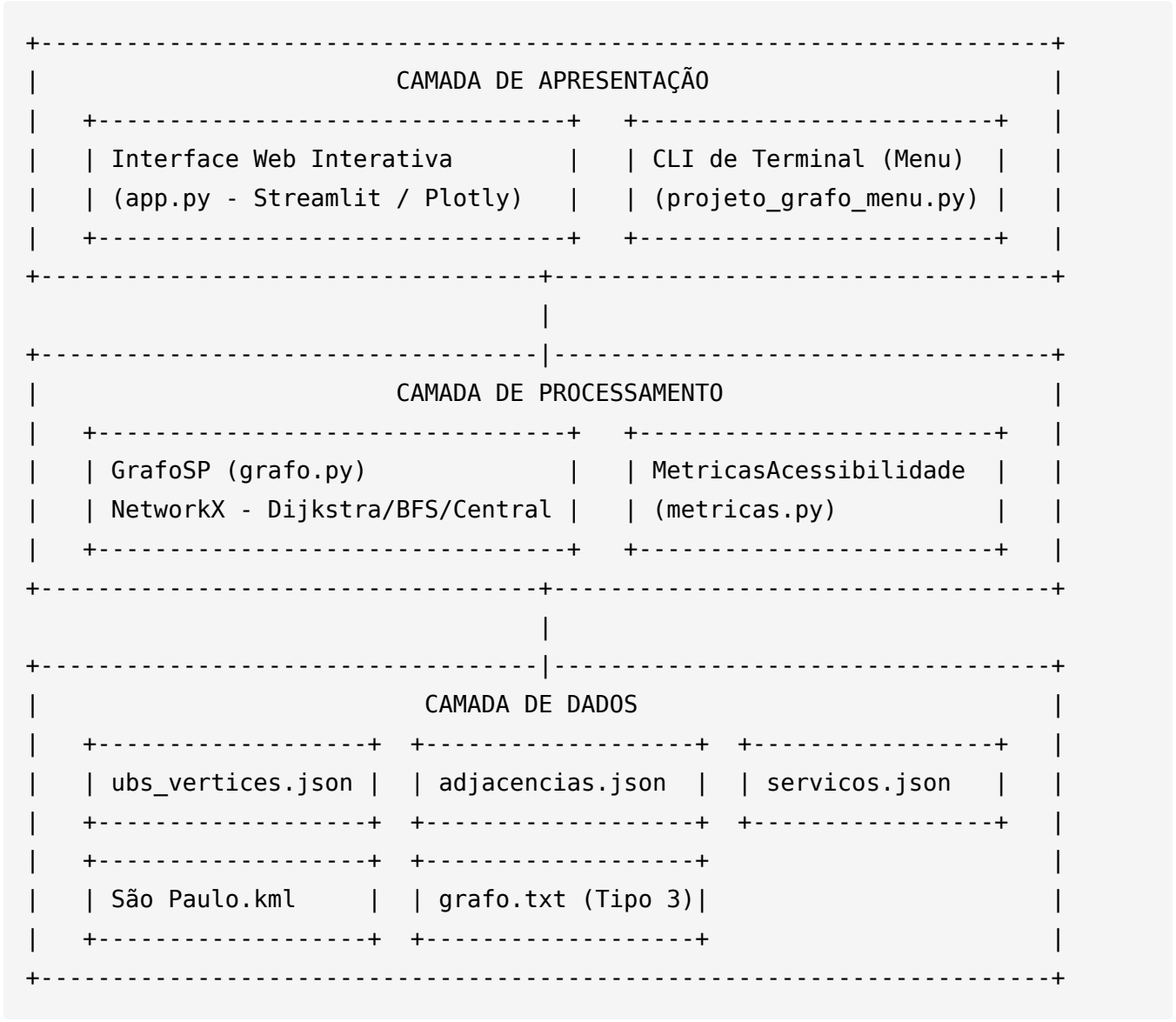
Equipamento de Saúde	Distância Média (km)	Distância Mediana (km)	Score Médio (S_i)	Distância Máxima (km)	Situação Territorial		
Unidades Básicas de Saúde (UBS)	$0{,}01$ km	$0{,}00$ km	**99,9 / 100**	$0{,}75$ km	**Excelente:** Cobertura Capilar Ampla		
Hospitais SUS	$1{,}80$ km	$1{,}11$ km	**91,0 / 100**	$15{,}74$ km	**Boa:** Concentração em Centros Regionais		
Unidades de Pronto Atendimento (UPA)	$15{,}72$ km	$16{,}42$ km	**30,7 / 100**	$39{,}86$ km	**Crítica:** Alta Desigualdade nas Periferias		

---

5. Arquitetura de Software e Módulos do Sistema

5.1 Visão Geral da Arquitetura

O sistema adota uma arquitetura em camadas modular e desacoplada em Python:


CAMADA DE APRESENTAÇÃO
Interface Web Interativa (app.py - Streamlit / Plotly)
CLI de Terminal (Menu) (projeto_grafo_menu.py)
CAMADA DE PROCESSAMENTO
GrafoSP (grafo.py) NetworkX - Dijkstra/BFS/Central
MetricasAcessibilidade (metricas.py)
CAMADA DE DADOS
ubs_vertices.json
adjacencias.json
servicos.json
São Paulo.kml
grafo.txt (Tipo 3)

5.2 Estrutura dos Arquivos do Código Fonte

```
Grafos/
├── src/
│   ├── __init__.py
│   ├── app.py                      # Dashboard Web Interativo (Streamlit + Plotly)
│   ├── grafo.py                    # Classe GrafoSP (NetworkX, Dijkstra, BFS, Centralidad
│   ├── metricas.py                 # Classe MetricasAcessibilidade (Ranking, Scores, Medi
│   ├── ihc.md                      # Documentação e avaliação de usabilidade IHC
│   └── RELATORIO_QUESTIONARIO_IHC.md # Relatório de Testes de Usabilidade
├── data/
│   ├── distritos.json              # Metadados dos 96 distritos de SP
│   ├── ubs_vertices.json           # Vértices (71 UBSs mantidas na amostra)
│   ├── adjacencias.json            # 215 Arestas ponderadas por distância (km)
│   ├── servicos.json               # Cadastro filtrado de UBS, UPA e Hospitais SUS
│   ├── deinfosacadsau2014.csv      # Dataset real de estabelecimentos de saúde
│   ├── evolucao_msp_pop_sexo_idade.csv # Dataset real de população distrital
│   └── São Paulo.kml               # Polígonos vetoriais geográficos da cidade
├── projeto_grafo_menu.py           # Aplicação CLI com menu textual (Opções a-j)
├── gerar_dados.py                  # Script ETL de limpeza e preparação de dados
├── gerar_grafo_txt.py              # Exportador para o formato padrão grafo.txt
├── grafo.txt                       # Grafo serializado no padrão exigido (Tipo 3)
├── README.md                       # Documentação principal
└── requirements.txt                # Dependências Python
```

5.3 O Formato de Arquivo Padronizado `grafo.txt`

A aplicação suporta leitura e gravação no formato de texto estruturado:

```
3                    <-- Tipo do Grafo (3: Peso no nó e na aresta, Não-orientado)
71                   <-- Número de Vértices (n)
1 "UBS JARDIM JAQUELINE" 45210.00
2 "UBS CHÁCARA DO CONDE" 38920.00
...                  <-- [ID] "[RÓTULO/NOME]" [PESO_NÓ (POPULAÇÃO)]
215                  <-- Número de Arestas (m)
1 2 3.42
1 5 1.85
...                  <-- [NÓ_ORIGEM] [NÓ_DESTINO] [PESO_ARESTA (DISTÂNCIA KM)]
```

5.4 Menu Textual da Interface CLI (`projeto_grafo_menu.py`)

A aplicação de terminal oferece um menu interativo com 10 opções funcionais:

```
==================================================
        MENU - PROJETO TEORIA DOS GRAFOS (PARTE 2)
==================================================
a) Ler dados do arquivo grafo.txt
b) Gravar dados no arquivo grafo.txt
c) Inserir vértice
d) Inserir aresta
e) Remover vértice
f) Remover aresta
g) Mostrar conteúdo do arquivo
h) Mostrar grafo (lista de adjacência)
i) Apresentar conexidade do grafo e grafo reduzido
j) Encerrar aplicação
==================================================
```

---

6. Avaliação de Interação Humano-Computador (IHC)

A interface gráfica web (`src/app.py`) foi desenvolvida sob o rigor das **10 Heurísticas de Usabilidade de Jakob Nielsen**:

1. **Visibilidade do Estado do Sistema:** Métricas em tempo real no topo da tela indicando distância de referência, índice de cobertura, posição no ranking e situação relativa (Acima, Na Média, Abaixo da Média).
2. **Correspondência entre o Sistema e o Mundo Real:** Uso de terminologia geográfica real (nomes de distritos, zonas Leste/Norte/Sul, terminologia SUS) e mapas vetoriais familiares (KML de São Paulo).
3. **Controle e Liberdade do Usuário:** Filtros dinâmicos na barra lateral para troca de UBS/Distrito de origem e alternância entre tipos de serviço de saúde (UBS, UPA, Hospital SUS).
4. **Consistência e Padrões:** Paleta de cores padronizada por zona geográfica (Norte: Azul `#3498db`, Sul: Verde `#2ecc71`, Leste: Laranja `#f39c12`, Centro: Vermelho `#e74c3c`, Oeste: Roxo `#9b59b6`).
5. **Prevenção de Erros:** Tratamento gracioso de exceções no cálculo do Dijkstra quando não há caminhos e pré-carregamento em cache (`@st.cache_resource`) dos objetos do grafo.
6. **Reconhecimento em Vez de Mapeamento:** Mapas e gráficos Plotly interativos com *tooltips* explicativos sobre cada nó e aresta.
7. **Flexibilidade e Eficiência de Uso:** Visualizações em abas organizadas (*Visão Geral*, *Análise da UBS*, *Cobertura/Ranking*, *Sobre*).
8. **Estética e Design Minimalista:** Interface limpa, moderna, sem poluição visual, destacando métricas quantitativas.
9. **Auxílio aos Usuários na Recuperação de Erros:** Mensagens informativas claras caso arquivos de dados estejam ausentes ou necessitem de regeneração.
10. **Ajuda e Documentação:** Aba dedicada ("Sobre") detalhando a fundamentação teórica, fórmulas matematicamente aplicadas, ODS 10 e equipe.

---

7. Resultados Obtidos e Discussão dos Achados

7.1 Síntese Estatística da Rede Urbanística

- **Total de Vértices ($|V|$):** 71 UBSs/Distritos.
- **Total de Arestas ($|E|$):** 215 conexões territoriais.
- **Grau Médio dos Vértices:** $6{,}06$ conexões por distrito.
- **Densidade:** $0{,}0865$ ($8{,}65\%$).
- **Componentes Conexos:** 1 (Grafo Inteiramente Conexo).

7.2 Diagnóstico de Cobertura por Serviço de Saúde

1. **Rede de Atenção Básica (UBS):** Presente na totalidade dos distritos ou a distâncias de adjacência insignificantes ($d_{\text{média}} = 0{,}01\ \text{km}$), demonstrando excelente descentralização territorial.
2. **Rede Hospitalar (Hospitais SUS):** Apresenta boa distribuição média ($d_{\text{média}} = 1{,}80\ \text{km}$), mas com bolsões de isolamento nos extremos periféricos (ex: distritos com distâncias superiores a $10\ \text{km}$).
3. **Rede de Prontos Atendimentos (UPA):** Apresenta o cenário mais desafiador ($d_{\text{média}} = 15{,}72\ \text{km}$), com scores médios de cobertura caindo para $30{,}7/100$, indicando a necessidade de novos aportes de infraestrutura em regiões periféricas estratégicas.

---

8. Conclusões e Trabalhos Futuros

8.1 Conclusão

O projeto **SPGraph** demonstrou com êxito a aplicação prática da **Teoria dos Grafos** na solução de um problema real de saúde pública e planejamento urbano. A abordagem quantitativa por grafos permitiu:

- Identificar objetivamente gargalos e distritos isolados.
- Avaliar a centralidade estratégica de unidades de saúde.
- Oferecer uma ferramenta interativa e intuitiva voltada para a promoção da **ODS 10 (Redução das Desigualdades)**.

8.2 Proposta de Trabalhos Futuros

1. **Modelagem Multimodal de Mobilidade:** Incorporar dados do transporte público (SPTrans/Metrô) para calcular o tempo de viagem em minutos em vez de distância euclidiana/distrital em km.
2. **Algoritmos Avançados de Acessibilidade:** Implementar o método **2SFCA (Two-Step Floating Catchment Area)** para considerar simultaneamente a capacidade de atendimento dos médicos e a pressão demográfica local.

Relatório de apresentação gerado para o projeto SPGraph — São Paulo, 2026.